SNES Assembly Adventure 02: Basic Assembly Programming

02 Aug 2018

Update February 2022: All code examples from all articles in this series can now be found on Github in one repository

Welcome back, Adventurer. I hope you are ready to continue your quest. Last time we set up a development environment to write and test our SNES games. In this article, we will have a closer look at the 65816 microprocessor and how it works. Then we will write some simple game logic and analyze it.



You earned the Development Evironment Badge, yay!

Quick Refresher: Binary and Hexadecimal Numbering System

If you have some programming experience you're probably already familiar with binary and hexadecimal numbers. If not, watch this video as a quick refresher:

Here is a more detailed introduction from the excellent Z80 Heaven Wiki.

To distinguish binary and hexadecimal numbers I will from now on prefix binary numbers with the percent sign % and hexadecimal numbers with a hash #:

```
%00101010   ; binary
$2c     ; hexadecimal
```

This is also the way to declare numbers source code for the cc65 toolchain. So you will see this a lot from now on.

The 65816 Microprocessor

This is the heart of the SNES. Everything we do from now on will revolve around the 65816 16-bit microprocessor (at least, until we get to audio). The 65816 is the successor to the 6502 and 65C02 (an improved version of the 6502) 8-bit microprocessor. A highly successful and widespread microprocessor used in a range of computers like the Commodore 64 or the original NES. Actually, the 65816 instruction set is a superset of the 65C02. So (almost) any program

written for the 65C02 will also run on the 65816. Keep this in mind since it is important when we talk about emulation and native mode in a moment. I won't go too deep into the history of the 65816 - there are tons of resources on the web about it.

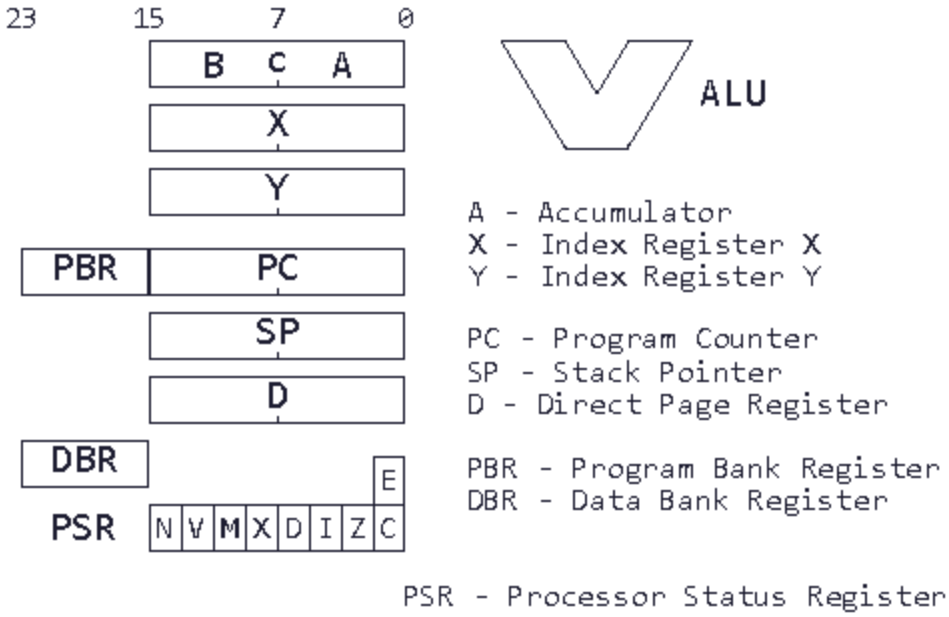
To be precise, the CPU in the SNES is a Ricoh 5A22, a custom chip developed by Nintendo that adds certain features to the 65816. We will use these features in later articles.

So, what can the 65816 do for us? It will take zero, one, or two operands and perform an operation (on them). Here is a short list of the operations it can perform:

- Arithmetic operations (addition and subtraction)
- Logical operations (AND, OR, XOR, right/left shift)
- Move data to/from memory
- Compare numbers
- Jump within the code

That's basically it. There are no high-level functions like `printf()` or `sqrt()`. It can't even multiply or divide! This is actually very important to understand when programming in assembly: A microprocessor does only very basic numbers crunching. Any high-level concepts like functions, strings, variables, etc. have to be implemented by the programmer.

Now, let's look at the basic architecture of the 65816:



65816 Architecture

We will refine this as we move along. The 65816 has three registers we can use: A, X, and Y. A is called the Accumulator, X and Y the Index Registers. A register is a very fast piece of memory inside the microprocessor. One register can hold 16 bits, or two bytes (this is not entirely true; actually, we have to switch them to 16-bit first, more on this in a moment). X and Y will always have the same size, while the size of A can be set independently.

The most important register (aka, the one we're going to use most often) is the *Accumulator* or A register. Now, the name of the accumulator can vary depending on the situation. Most of the time, we will refer to it as accumulator A. Yet certain instructions explicitly use the accumulator as a 16-bit register regardless of whether the M flag (see below) is set or not. This might be a bit confusing now, so here is a quick list that applies most of the time:

- When we address the accumulator as A, we implicitly mean the whole accumulator whether it's set to 8- or 16-bit; A can also refer to the lower byte in the accumulator
- When we address the accumulator as B, we explicitly refer to the higher byte stored in the accumulator (bits 8 ~ 15)
- When we address the accumulator as C, we explicitly refer to the whole accumulator as a 16-bit register

Again, this will make a lot more sense once you learn more about how the 65816 operates. You might wanna come back to this at a later time.

So we have three registers we can use. If you think, "Well, that's not a lot", you're perfectly right. The 6502 and 65816 were notorious for having only three (working) registers. In contrast, the Motorola 68000 microprocessor (sometimes called 68k) used in the Sega Genesis/Mega Drive has 16 registers each 32 bits wide! Speak about what Nintendon't. If you're interested in why that is, you can read up on RISC and CISC microprocessors here.

Usually, the programmer will load certain values into the registers, and the microprocessor will operate on these values. It is important to note that not every operation can be performed on every register. For example, we can only add a number to the number in register A (hence the name *Accumulator*). In fact, most arithmetic or logic operations are limited to the accumulator. But more on that later.

Next, there are six special purpose registers:

- **Direct Page Register (D)**: used for direct page addressing, holds 16 bits
- **Data Bank Registers (DBR) and Program Bank Register (PBR)**: used for addressing memory, hold 8 bits each
- **Stack Pointer (S)**: points beyond the last item pushed to the stack, holds 16 bits
- **Program Counter (PC)**: holds the address of the current instruction to execute, holds 16 bits
- **Processor Status Register (SPR)**: holds the state of the processor after the last instruction

We will discuss each register in detail when appropriate. For now, we will only look at the Processor Status Registers. Every bit within it represents a certain state of the processor:

- **N**: Negative flag. Is set when the result of the last operation is a negative number (i.e., the most significant bit of the result is set)
- **V**: Overflow flag. Is set when the last operation results in an overflow.
- **M**: Memory/Accumulator Select flag. Controls the size of register A, the accumulator. If set, the accumulator will be 8-bit, else 16-bit.
- **X**: Index Register Select flag. The same as the Memory/Accumulator Select flag, but for the X and Y registers.
- **D**: Decimal Mode flag. Will select whether the 65816 operates in decimal mode. This is disabled for the SNES, so you don't need to care about this flag.
- **I**: IRQ Disable flag. Controls whether the processor will react to an interrupt request. Interrupts will be covered in a later article. Think of them as external requests to the processor to execute a certain procedure/function.

- **Z**: Zero flag. Is set if the result of the last operation was zero.
- **C**: Carry flag. Is set if a carry occurred during the last operation.
- **E**: Emulation flag. This controls whether the processor is operating in emulation or native mode.

Generally, we say a flag is set when it is 1, and clear when it is 0. You might wonder why the Emulation flag is drawn above the Carry flag. This is because we cannot access the Emulation flag directly.

Emulation and Native Mode

Earlier I told you the 65816 instruction set is a superset of the 65C02 instruction set. When we turn on/reset the 65816 it starts in emulation mode. In emulation mode, the 65816 acts like its predecessor the 65C02. This means we can only use the instruction set of the 65C02 and not the extended 65816 instructions. This feature was meant to guarantee backward compatibility. If we want to use the full functionality of the 65816 (new addressing modes, 24-bit address bus, etc.) we first have to switch to native mode. One thing important to remember here: while in emulation mode, the registers of the 65816 are only 8 bits wide, not 16! So once the 65816 has started, we first need to switch it from emulation to native mode. And then we need to tell it explicitly that we want to use 16-bit registers by manipulating the M and X flags of the processor status register. So keep in mind: The 65816 starts in emulation mode. To use its full 16-bit powers, we need to switch to native mode first.

This might sound complicated, but actually, it only takes three simple instructions. We will not do this in this article yet, but in the next ones, we will start wielding the full 16-bit powers of the 65816. For now, think that A, X, and Y can hold a byte each.

A crucial part of any microprocessor is the ALU, the Arithmetic Logic Unit. It handles all arithmetic and logical operations. The exact inner workings of it are not important to us. All you need to know is that it will execute the instructions and update the processor status register accordingly.

Those are the internal components of the 65816 microprocessor. To communicate with other parts of the system it uses two buses:

- The Address Bus: this bus is 24 bits wide, the 65816 can address up to 16 Megabytes
- The Data Bus: this bus is 8 bits wide, this bus actually moves data between the processor and memory.

(*Note*: This isn't entirely accurate; in reality, the 65816 has only 16 address and 8 data pins. The 65816 utilizes a technique called *multiplexed bus* where the 8 data pins are used both for the data and address bus. But this happens on the hardware level, we as programmers don't have to concern ourselves with this. The logic inside the SNES takes care of this.)

Whenever the processor wishes to load or store data in memory, it will first put the address on the address bus and then read or write the data through the data bus. We will look at this process in more detail shortly.

If you wonder why the architecture overview above shows only a 16-bit address bus, here is why. The 65816 has a total of 24 addressing modes (you might find slightly different numbers in other sources; e.g., some do not count *Absolute Indexed X* and *Absolute Indexed Y* as distinct addressing modes, others do. But don't mind that yet. You only need to understand the differences between them, then the total number doesn't really matter). In general, an addressing mode is the way a processor calculates the final address (called the *effective address*) of an operation. When in native mode, the 65816 will use the Data and Program Bank Register to extend the 16-bit address bus to a 24-bit bus. This might sound confusing now, and understanding every single addressing mode takes some time. Later in this article, you will learn your first two addressing modes, *immediate* and *absolute* addressing (mode). I will cover each addressing mode in more detail as we move along. Understanding the addressing modes of a processor is key to writing effective and tight assembly code (you will read this sentence *a lot* from me).

Before we proceed, let's take a second and summarize what we know so far about the 65816 microprocessor:

- It has three working registers:
    - The Accumulator, or A register
    - The Index Registers, X and Y
- It has six special purpose registers:
    - Data and Program Bank Registers
    - Direct Page Register
    - Stack Pointer
    - Program Counter
    - Processor Status Register
- A 24-bit address bus to address up to 16 Megabytes
- An 8-bit data bus to write or read data to/from memory

This general overview should be enough for now. We will return to each register in more detail when we cover its function and purpose. Let's finally get down to business and write some code.

This will clarify some of the 65816 architecture's details.

A Simple Introduction to 65816 Assembly

When programming in assembly, two concepts are key to writing fast and efficient assembly code: Understanding the microprocessor's addressing modes, and how each operation affects the processor status register. I have described the processor status register in general above. I will explain each flag in more detail as we proceed through this series of articles. To this end, I will introduce you to the various instruction codes and addressing modes of the 65816 one by one. Also, remember that the 65816 starts in emulation mode. So for now, registers can only hold 8 bits, not 16.

Your First Opcodes

Microprocessor (machine) instructions are called *opcodes*. The shortcuts to represent these opcodes we use in assembly source code are called *mnemonics*. In most assembly languages mnemonics consist of a two, three, or four letter abbreviation of the instruction/opcode it represents. Note that these two terms are often used interchangeably. This isn't entirely correct, there is a distinction between opcodes and mnemonics. But I will mainly use the term *opcode* and make sure to point it out if the distinction is of importance in the discussed context.

The very first two opcodes you will learn are the most basic (i.e., you will use them *all the time*): LDA will load a value from memory into the accumulator. STA will store the content of the accumulator in memory:

```
lda : LoaD Accumulator
sta : STore Accumulator
```

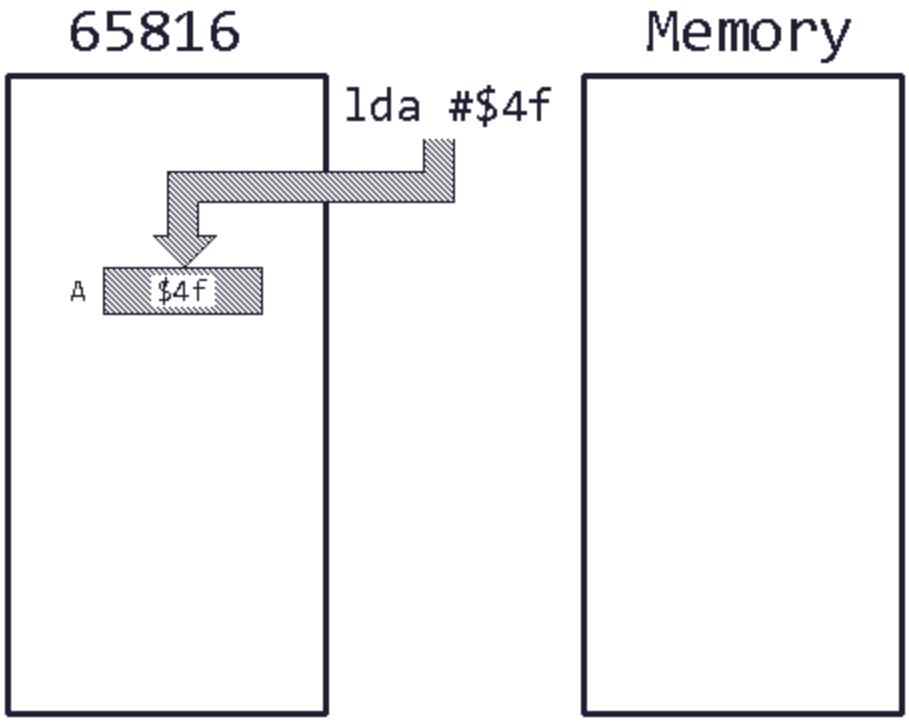
The 65816 has a total of 24 addressing modes. The first two are called *immediate* and *absolute addressing* modes.

Immediate addressing is used for data that is constant throughout the program. That means the value loaded into a register is not taken from memory but from a constant. We prefix the value we load into a register with a hash mark (#) to signal immediate addressing:

```
1  ; immediate addressing mode examples
2  lda #100        ; load the value 100 into A
3  lda #$ff        ; load the hex value $ff into A, which is equal to 255 decimal
```

ImmediateExample.asm hosted with ❤ by **GitHub** **view raw**

Here's a graphical representation of *immediate addressing*:



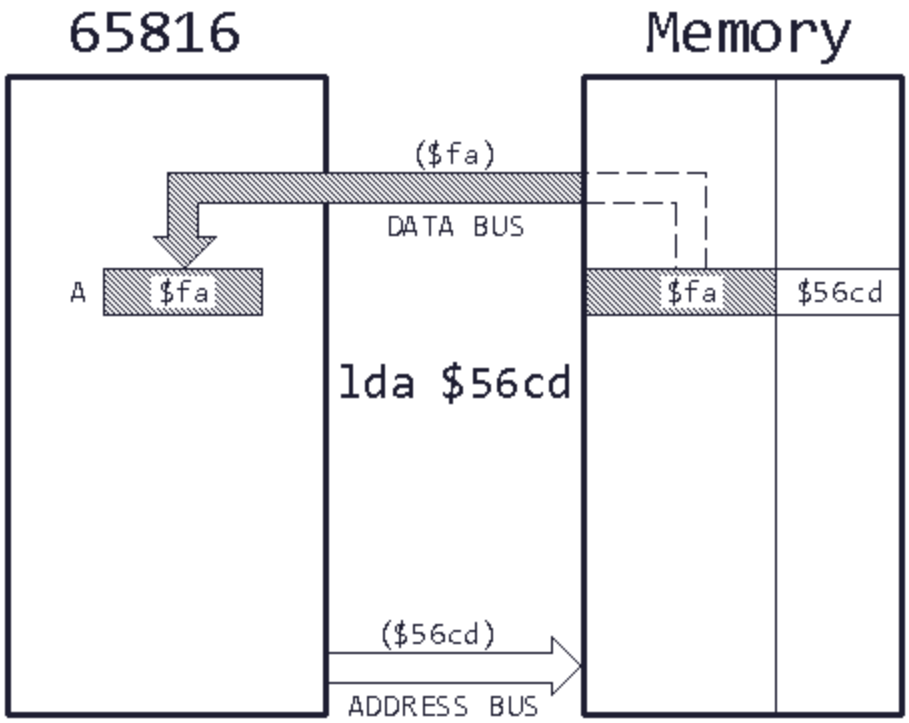
Immediate Addressing

In *absolute addressing* mode, we tell the opcode explicitly where to load from or store the data in the register. Unlike immediate addressing this actually moves data from or to memory:

```
1  ; absolute addressing mode examples
2  lda #100          ; load the value 100 into A
3  sta $0001         ; store the value in A (100) at memory location $0001
4  lda $000a         ; load the value at memory location $000a into A
```

AbsoluteExample.asm hosted with ❤ by **GitHub** **view raw**

Here's a graphical representation of *absolute addressing*:



Absolute Addressing

You might be wondering why we use 16-bit addresses even though I told you the 65816 has a 24-bit address bus. This is because the 65816 calculates the final address (i.e., the *effective address*) by combining the address given by the opcode and the Data or Program Bank Register. That's pretty much what the different addressing modes are all about: how to calculate the *effective addressing* the current operation will execute on. In the next article, I'll introduce you to your first 24-bit addressing mode.

The high byte of a 24-bit address is often called the address bank, while the middle and low bytes are called the address offset. For better readability, we separate the bank and offset address by a colon: $01:1A53 is the same as $011A53. The Data Bank Register and Program Bank Register are set to $00 on startup/reset. Those registers can be manipulated by special instructions only. For now, we will only use the memory space from $00:0000 to $00:FFFF, which equals 64 Kilobytes or one page or bank of memory (i.e., they can be accessed completely with 16-bit addresses).

More Opcodes

Now, only loading and storing data won't get us very far. So let's introduce four more opcodes, even one that actually manipulates register data:

```
cmp : CoMPare accumulator
bcc : Branch if Carry Clear
bcs : Branch if Carry Set
clc : CLear Carry flag
adc : ADd with Carry
```

The first one, CMP, will compare the value in the accumulator to another. CMP again can use immediate or absolute addressing mode:

```
1  ; example showing CMP opcode with different addressing modes
2  lda #$80        ; load the value $80 into A
3  cmp #$a0        ; compare A to $a0
4  lda #$44        ; load the value $44 into A
5  cmp $affe       ; compare A to the value stored in memory location $affe
```

CMPExample.asm hosted with ❤ by **GitHub** **view raw**

Earlier I told you about the importance of understanding how opcodes affect the processor status register. CMP will set or clear the carry flag depending on the result: If the value in the accumulator *is smaller* than the value we compare it to, the carry flag will be clear. If the value in the accumulator is *equal to or greater* than the compare value, the carry flag will be set. This behavior can be used to implement something similar to conditional expressions or if-else clauses.

Enter your first branch instructions, BCC. This opcode will check whether the carry flag is set or clear. If it is clear, the program will branch (i.e., jump) to the label/address specified in the opcode and continue execution from there. Labels are a useful tool to make our code more readable. Instead of using fixed addresses like $124A, we let the assembler replace our labels with the actual address.

BCS works the exact same way, but it will branch if the carry flag is set. We will see an example of this in a moment.

For now, think of labels as an alias for a given address or number.

Let's clarify this with a simple example. Say we want to check whether the value in the accumulator is greater than 64:

```
; simple example for comparing two values
lda #$80            ; load the value $80/128 into A
cmp #$40            ; compare A to $40/64, this will set the C(arry) flag since
bcs GreaterThan     ; if the C(arry) flag is set, jump to GreaterThan label
sta $0001           ; this code will never be executed
GreaterThan:
sta $0002           ; store A in memory location A
```

SimpleCMPExample.asm hosted with ❤ by **GitHub** **view raw**

In the above example, we first load the value $80 in the accumulator. Then we compare it to $40. Since $80 is greater (or equal) than $40, the CMP instruction will set the carry flag to signal that. Next, BCS checks whether the carry flag is set. If so, the program will jump to the label specified after the instruction. If not, the program will simply continue and execute the next instruction. In this example, the `sta $0001` instruction is never executed because the BCS instruction will cause the program to jump to the `GreaterThan` label and continue execution with `sta $0002`.

This might be a bit strange to wrap your head around if you're used to other programming languages like C or Python. But don't despair, once you get more experienced with 65816 code, you will get used to this very quickly.

Don't worry about whether you remember which opcode effects which processor flags. As you learn new opcodes and addressing modes you will notice the logic behind them and will be able to tell which flag is affected by simply looking at the code (for all other cases there is a cheat sheet I will show you in a later article).

Simple Addition

The next two opcodes are almost always used together. The first, CLC, clears the carry flag. So after the opcode is executed the carry flag will be cleared to 0. That's it.

Now, ADC, or *ADd with Carry*, will execute an addition on the accumulator. It will take the value provided to the opcode and add it to the value already stored in register A. Again we can either use immediate or absolute addressing mode:

```
; add a number to A and store in in memory
lda #$20        ; load the value $20/32 into A
clc             ; clear the carry flag
adc #$0a        ; add $0a to A: $20 + $0a = $2a/42
clc             ; clear the carry flag
adc $0020       ; add the value at memory location $0020 to the value in A
```

ADCExample.asm hosted with ❤ by **GitHub** **view raw**

Why do we need to clear the carry flag before an addition? The reason is that to get a correct result from a binary addition we need to clear the carry flag beforehand. The ADC opcode will actually always add the carry to the result of the addition. This might seem to be a weird behavior (and in fact, not all processor architectures do that) but once we get to 16-bit operations, it will make a lot more sense (if we want to conduct 16-bit additions, we need a way to transfer the carry from the lower to the higher byte).

If you need a refresher on binary arithmetic, read this.

You now know six opcodes and two addressing modes. These are enough to write some simple game logic, as we will do now. Keep in mind that there are more addressing modes to come and not every opcode can utilize every addressing mode.

Some Simple Game Logic

Let's finally write some useful code. Say we want to check whether the player has collected 100 coins and therefore gains an extra life. In C it might look like this:

```
1   int coins = 0;      // the player starts with 0 coins...
2   int lives = 3;      // ...and 3 lives
3
4   // ...some game code...
5
6   // now the player has collected a coin
7   if(coins == 100)    // did the player collect 100 coins?
8   {
9       coins = 0;          // reset the coin counter to zero
10      lives = lives + 1;  // increase the number of lives by one
11  }
12
13  // ...more game code...
```

extralife.c hosted with ❤ by **GitHub** **view raw**

Pretty straightforward. Now, let's do the same in assembly:

```
1   ; We initalize two "variables" with zero and three.
2   ; We will use the memory locations $00:0000 and $00:0001
3   ; to store the number of coins and lives.
4       lda #$00        ; load the A register with zero...
5       sta $0000       ; ...and store in memory at $00:0000
6       lda #$03        ; load the A register with three...
```

```
 7       sta $0001       ; ...and store in memory at $00:0001
 8
 9   ; ...here goes some game code...
10
11   ; now let's check the number of coins
12       lda $0000       ; load the number of coins into the accumulator
13       cmp #100        ; compare the number in accumulator to 100
14       bcc Done        ; if the number in accumulator is less than 100, jump to do
15       lda #$00        ; else, load the accumulator with zero...
16       sta $0000       ; ...and reset the coin counter
17       lda $0001       ; next, get the current number of lives...
18       clc             ; ...clear the carry flag...
19       adc #$01        ; ...and add one to the number in A
20       sta $0001       ; store the new number of lives
21   Done:               ; all done
22
```

extralife.asm hosted with ❤ by **GitHub** **view raw**

Wow, this looks way more complicated. Let's have a closer look.

Lines 4 through 7: First, we arbitrarily choose two memory locations to store the number of coins. For simplicity, we choose $00:0000 for the number of coins and $00:0001 for the number of lives. Next, we store the starting values. The player starts with 0 coins and 3 lives.

Lines 12 through 14: This is the crucial part of this example. These three opcodes implement a behavior similar to a conditional statement or if-clause. First, we load the number of coins into the accumulator. And then compare it to 100. As explained earlier, the CMP opcode will modify the carry flag: If the value in the accumulator is smaller than 100, the carry flag will be clear, else it will be set. Next, BCC will check whether the carry flag is clear. If it is (so the number of coins is less than 100) the program will branch to the Done label and skip the code in lines 15 through 20. If the carry flag is set (so the number of coins is equal to or greater than 100), then nothing happens and the program continues execution at line 15.

Lines 15 through 20: This part of the code will reset the number of coins to zero and increase the number of lives by one. This is pretty straightforward. We load the accumulator with the value of $00 and store it in memory at $00:0000 where we keep track of the number of coins. Next, we load the current number of lives into the accumulator. Then we clear the carry flag in preparation for the addition. We add $01 to the value in the accumulator, and finally, store the new number of lives back into memory at $00:0001.

I hope this simple example wasn't too hard to follow. If you have any questions, use the comment function below and I'll try and help.

Now, this code is really hard to read. There are a lot of numbers that can easily be confused. It is not directly clear what they do. Let us improve this code with labels to make it more readable.

Improving the Code with Labels

Labels are a convenient way to improve the readability of assembly code. We will replace the memory locations where we store the number of coins and lives with labels:

```
1  ; tell the assembler we want to use two labels
2      coins = $0000
3      lives = $0001
4
5  ; We will use the memory locations $00:0000 and $00:0001
6  ; to store the number of coins and lives.
7      lda #$00        ; load the A register with zero...
8      sta coins       ; ...and store in memory at $00:0000
9      lda #$03        ; load the A register with three...
10     sta lives       ; ...and store in memory at $00:0001
11
12 ; ...here goes some game code...
13
14 ; now let's check the number of coins
15     lda coins       ; load the number of coins into the accumulator
16     cmp #100        ; compare the number in accumulator to 100
17     bcc Done        ; if the number in accumulator is less than 100, jump to do
18     lda #$00        ; else, load the accumulator with zero...
19     sta coins       ; ...and reset the coin counter
20     lda lives       ; next, get the current number of lives...
21     clc             ; ...clear the carry flag...
22     adc #$01        ; ...and add one to the number in A
23     sta lives       ; store the new number of lives
24 Done:               ; all done
25
26 ; ...some more game code...
```

ExtraLifeImproved.asm hosted with ❤ by **GitHub** **view raw**

This looks better than before. The assembler (as we will see in more detail in a later article) will replace all instances of `coins` with $0000, and `lives` with $0001. This also demonstrates another advantage of labels: Say we later in the development cycle determine that we need to

move the memory location of the values of coins and lives. The only thing we need to change is the labels to accommodate the new memory locations without touching the rest of the code.

Conclusion

This concludes this section and article about basic 65816 assembly programming. I hope this wasn't too dry. Some concepts like addressing modes can be quite confusing to the beginner (but again, they are crucial). Later articles will go into more details about the data and program bank register, how they affect addressing, and how to manipulate them.

In the next article, we will create and actually display a sprite on the SNES.

As always, if you have questions or need any clarifications, please use the comment function below and I'll try and help.

References and Links

- If you want to jump deeper into 6502/65816 assembly, I recommend you check out Easy 6502. It's a simple introduction to 6502 assembly. Pretty much all concepts presented there are useful for SNES development, so go read it!
- Check out this complete overview of all 65816 opcodes. This is a very extensive but complete overview. I use it all the time as a reference while programming. It has also additional information on the two addressing modes we have discussed so far.
- Here are some other introductions to 65816 assembly programming:
    - Learning 65816 Assembly
    - The rather extensive Ersanio's ASM Tutorial, this goes beyond the basics and touches already on some SNES specific points.
    - Introduction to Assembly Programming on the Apple IIgs is a video series that shows the basics of 65816 assembly. Yes, the Apple IIgs used the same CPU as the SNES. Again, this goes beyond basics at a certain point but you can still learn a bit about instructions and addressing modes.
- All SNES Assembly Adventure code examples of this series on Github